

**Practical – 25: Install Flutter SDK, configure the development environment and display “Hello World” in the centre of the screen.**

**Install the Flutter and Dart plugins:**

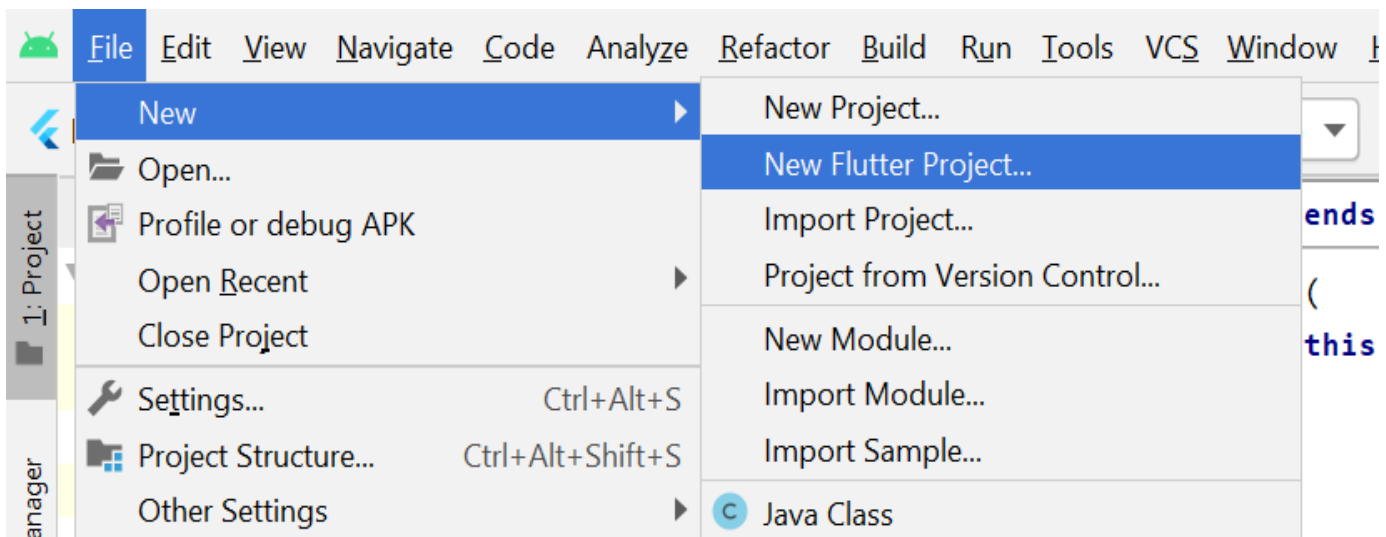
After the successful installation of Android Studio, you have to install Flutter and Dart plugins. To do so follow the steps mentioned below:

1. Start Android Studio.
2. Open plugin preferences (Configure > Plugins as of v3.6.3.0 or later).
3. Select the Flutter plugin and click Install.
4. Click Yes when prompted to install the Dart plugin.
5. Click Restart when prompted.

**Creating the application:**

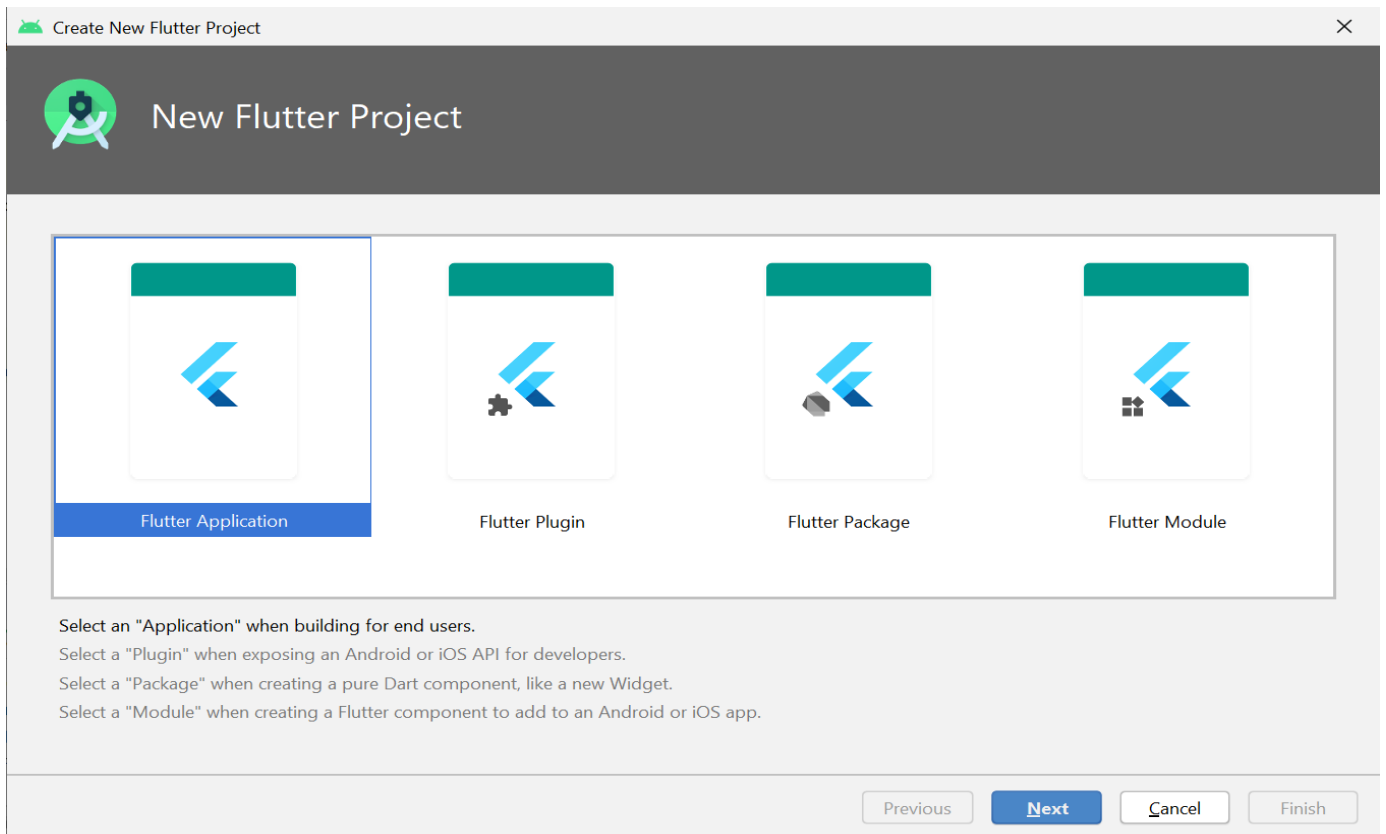
After installing Dart and Flutter plugins create a flutter app to check if it is working properly or not, to do so follow the steps mentioned below:

**Step 1: Open the IDE and select Start a new Flutter project.**



Select New Flutter Project from the file menu

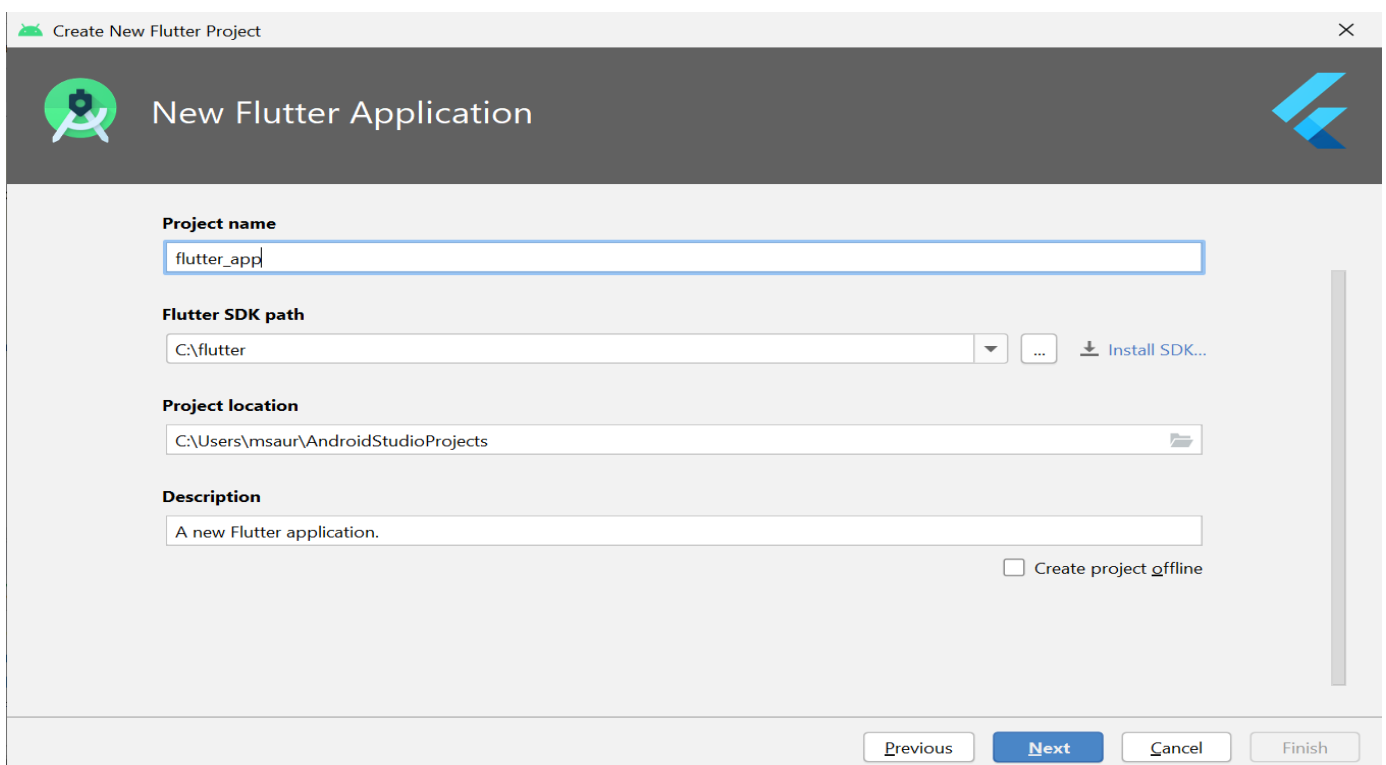
**Step 2: Select the Flutter Application as the project type. Then click Next.**



Select the type of flutter application

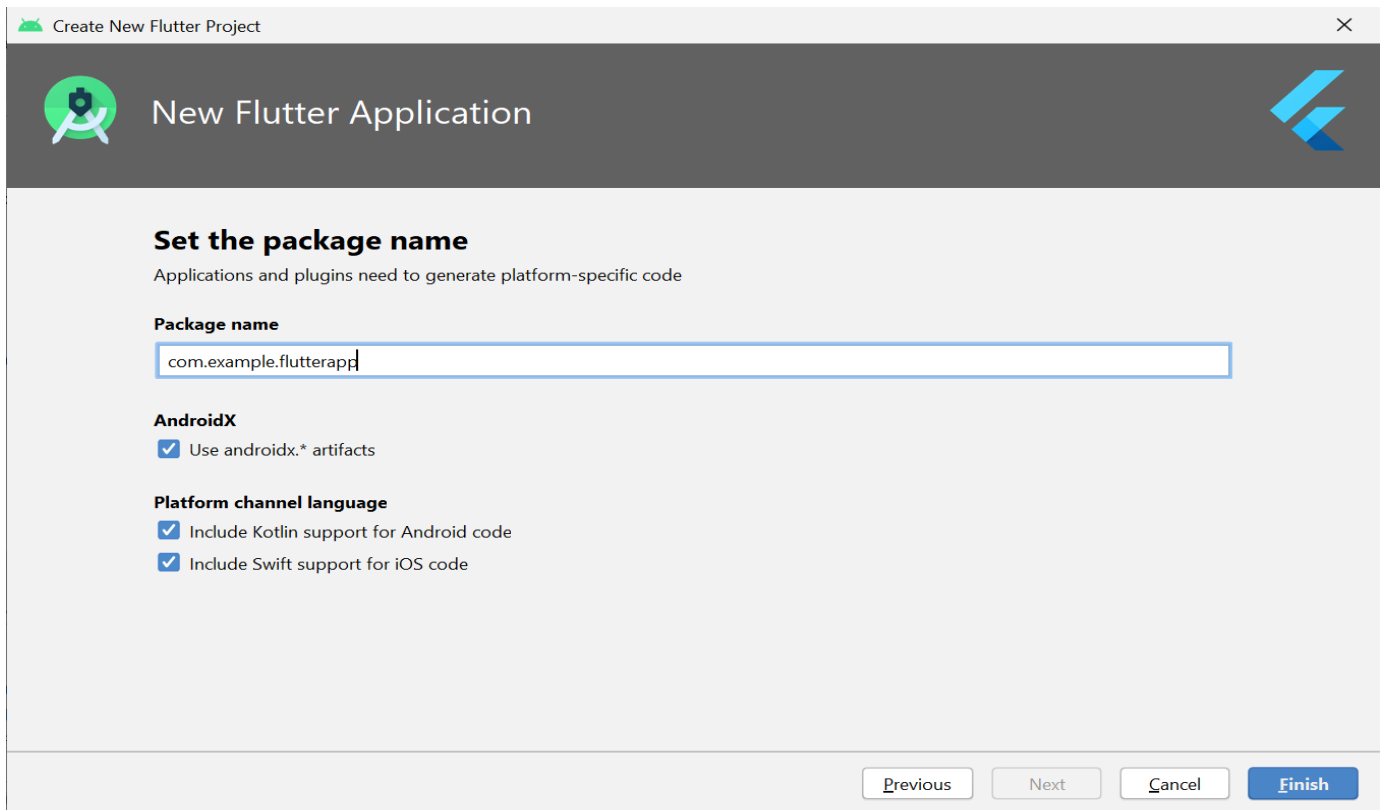
Step 3: Verify the Flutter SDK path specifies the SDK's location (select Install SDK... if the text field is blank).

Step 4: Enter a project name (for example, myapp). Then click Next.



Enter the name of the project and also its description. Also, add the path of the Flutter SDK in the Flutter SDK path or click on install SDK.

Step 5: Click Finish.



Create New Flutter Project

## New Flutter Application

**Set the package name**  
Applications and plugins need to generate platform-specific code

**Package name**  
com.example.flutterapp

**AndroidX**  
☒ Use androidx.\* artifacts

**Platform channel language**  
☒ Include Kotlin support for Android code  
☒ Include Swift support for iOS code

Previous Next Cancel **Finish**

Hit the Finish Button and wait till the project is created by the IDE.

Step 6: Wait for Android Studio to install the SDK and create the project.

Program Code:

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(const MyApp());  
}
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Flutter Demo',  
      theme: ThemeData(  

```

```
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),  
        useMaterial3: true,  

```

```
    ),  
    home: const MyHomePage(title: 'Flutter Demo Home Page'),  
  );  
}  
}
```

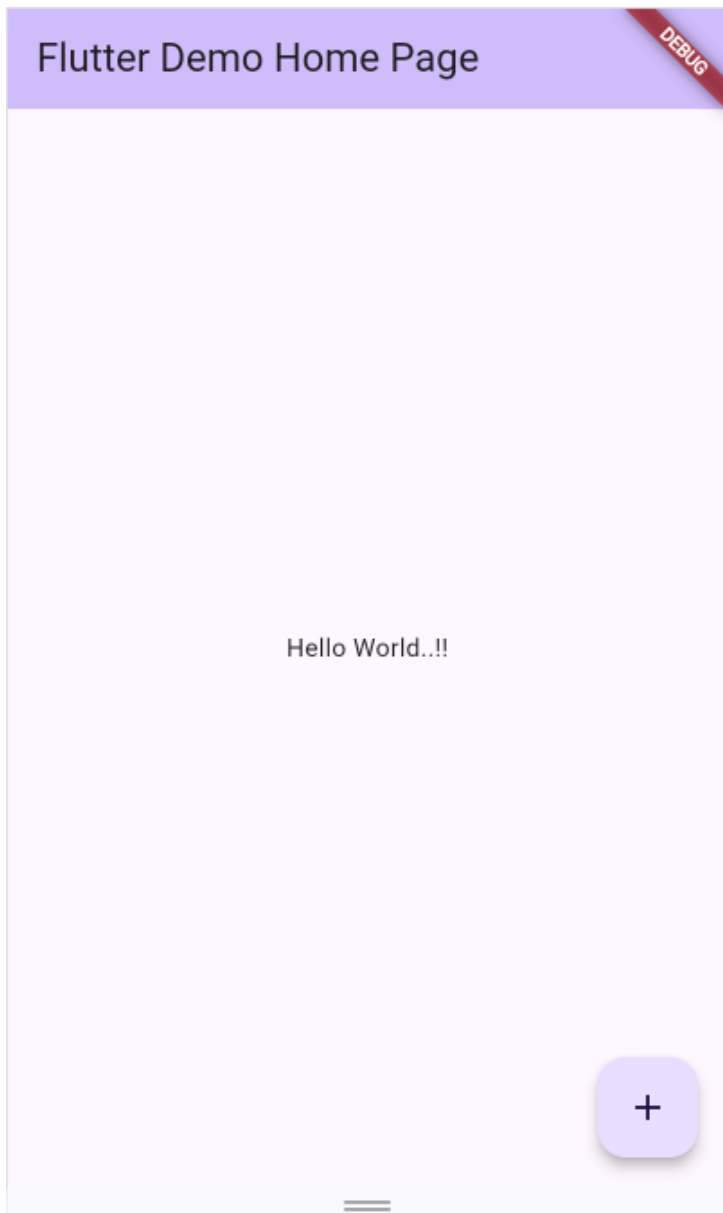
```
class MyHomePage extends StatefulWidget {  
  const MyHomePage({super.key, required this.title});  
  
  final String title;  
  
  @override  
  State<MyHomePage> createState() => _MyHomePageState();  
}
```

```
class _MyHomePageState extends State<MyHomePage> {  
  int _counter = 0;  
  
  void _incrementCounter() {  
    setState(() {  
      _counter++;  
    });  
  }  
}
```

```
@override  
Widget build(BuildContext context) {  
  
  return Scaffold(  
    appBar: AppBar(  
  
      backgroundColor: Theme.of(context).colorScheme.inversePrimary,  
  
      title: Text(widget.title),  
    ),  
    body: Center(  
  
      child: Column(  
  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: <Widget>[  
          const Text(  
            'Hello World..!!',  
          )  
        ],  
      ),  
    ),  
    floatingActionButton: FloatingActionButton(  
      onPressed: _incrementCounter,
```

```
    tooltip: 'Increment',  
    child: const Icon(Icons.add),  
  ), // This trailing comma makes auto-formatting nicer for build methods.  
);  
}  
}
```

Output :





Practical – 26: Develop a Flutter app to get two numbers from the user and display addition on the screen after clicking a button.

main.dart

```
import 'package:flutter/material.dart';
void main() {
  runApp(const MyApp());
}
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
        useMaterial3: true,
      ),
      home: const MyHomePage(title: 'Flutter Demo Home Page'),
    );
  }
}
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});

  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}
class _MyHomePageState extends State<MyHomePage> {
  @override
  Widget build(BuildContext context) {
    final _num1 = TextEditingController();
    final _num2 = TextEditingController();
    final _tot = TextEditingController();
    int sum;

    return Scaffold(
      appBar: AppBar(
        backgroundColor: Theme.of(context).colorScheme.inversePrimary,
        title: Text(widget.title),
      ),
      body: Center(
        // Center is a layout widget. It takes a single child and positions it
```

*// in the middle of the parent.*

```
child: Column(  
  mainAxisAlignment: MainAxisAlignment.center,  
  children: <Widget>[  
    SizedBox(  
      width: 300,  
      child: TextField(  
        controller: _num1,  
        decoration: InputDecoration(  
          hintText: "Enter No-1:", border: OutlineInputBorder(),  
        )),  
    SizedBox(  
      width: 300,  
      child: TextField(  
        controller: _num2,  
        decoration: InputDecoration(  
          hintText: "Enter No-2:", border: OutlineInputBorder(),  
        ),  
      ),  
    ElevatedButton(  
      onPressed: () {  
        sum = int.parse(_num1.text) + int.parse(_num2.text);  
        _tot.text = sum.toString();  
      },  
      style: const ButtonStyle(  
        backgroundColor: MaterialStatePropertyAll<Color>(Colors.green),  
      ),  
      child: Text('Add'),  
    ),  
    TextField(  
      controller: _tot,  
      decoration: InputDecoration(  
        labelText: "Total",  
        labelStyle: TextStyle(fontSize: 15, color: Colors.grey.shade400),  
      ),  
    ),  
  ],  
,  
,  
,  
,  
,  
,  
);  
}
```

Output :

Flutter Demo Home Page

DEBUG

50

40

Add

Total  
90



Practical – 27 : Develop a Flutter app for Login using static data. If the User ID and Password are correct then clicking a login button should open a new screen showing Username at the center of the new screen.

Main.dart

```
import 'package:demo_pr_27/page2.dart';
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
        useMaterial3: true,
      ),
      home: const MyHomePage(title: 'Flutter Demo Home Page'),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});

  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  @override
  Widget build(BuildContext context) {
    final _username = TextEditingController();
    final _password = TextEditingController();
    final s = TextEditingController();
    var msg = [
      // list of text which the text get form here
      "",
      "Please Enter Valid Username and Password..!!",
    ];
  }
}
```

```
int count = 0;
```

```
return Scaffold(  
  appBar: AppBar(  
    backgroundColor: Theme.of(context).colorScheme.inversePrimary,  
    title: Text(widget.title),  
  ),  
  body: Center(  
    child: Column(  
      mainAxisAlignment: MainAxisAlignment.center,  
      children: <Widget>[  
        SizedBox(  
          width: 300,  
          child: TextField(  
            controller: _username,  
            decoration: InputDecoration(  
              hintText: "Enter Username:-", border: OutlineInputBorder(),  
            ),  
          ),  
        ),  
        SizedBox(  
          width: 300,  
          child: TextField(  
            controller: _password,  
            decoration: InputDecoration(  
              hintText: "Enter Password:-", border: OutlineInputBorder(),  
            ),  
          ),  
        ),  
        TextButton(  
          onPressed: () {  
            String user = (_username.text).toString();  
            String pass = (_password.text).toString();  
  
            if (user == "ABC" && pass == "123") {  
              Navigator.push(  
                context,  
                MaterialPageRoute(builder: (context) => page2()),  
              );  
            } else {  
  
              // set up the AlertDialog  
              AlertDialog alert = AlertDialog(  
                title: Text("Error..."),  
                content: Text("Please Enter Valid Username or Password..!!"),  
                actions: [],  
              );  
            }  
          }  
        ),  
      ],  
    ),  
  ),  
);
```

```

        // show the dialog
        showDialog(
          context: context,
          builder: (BuildContext context) {
            return alert;
          },
        );
      },
    ],
  ),
);
}
}

```

page2.dart

```

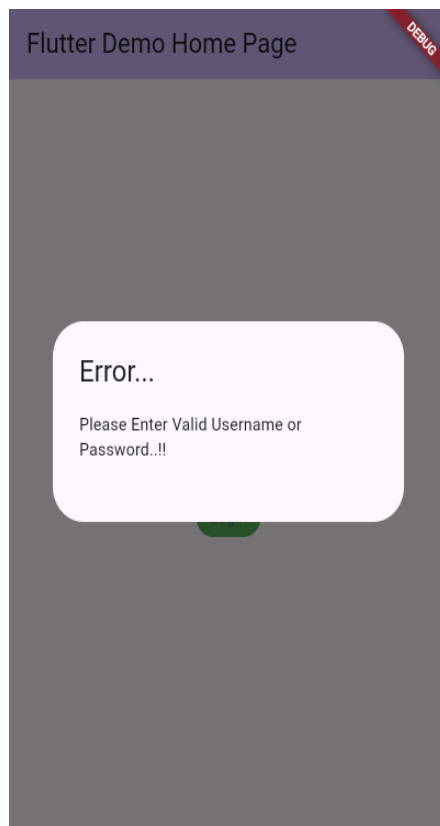
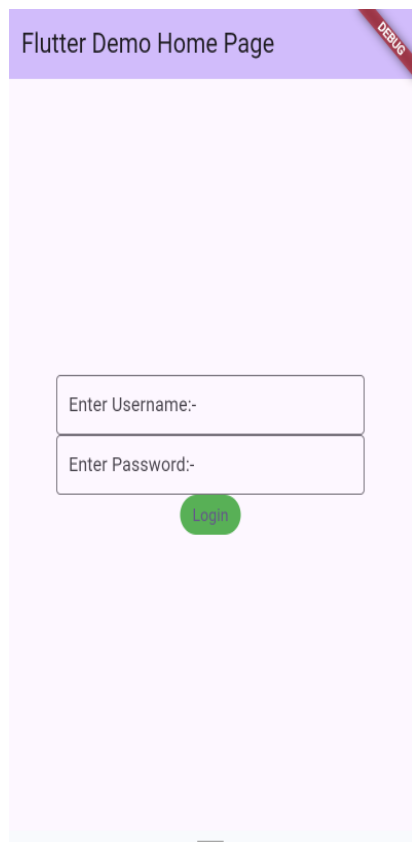
import 'package:flutter/material.dart';

class page2 extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Page 2"),
      ),
      body: Center(
        child: Column(
          children: <Widget>[
            Text(
              style: TextStyle(
                color: Colors.red,
              ),
              "Welcome to page 2"
            ),
          ],
        ),
      ),
    );
  }
}

```

```
}  
}
```

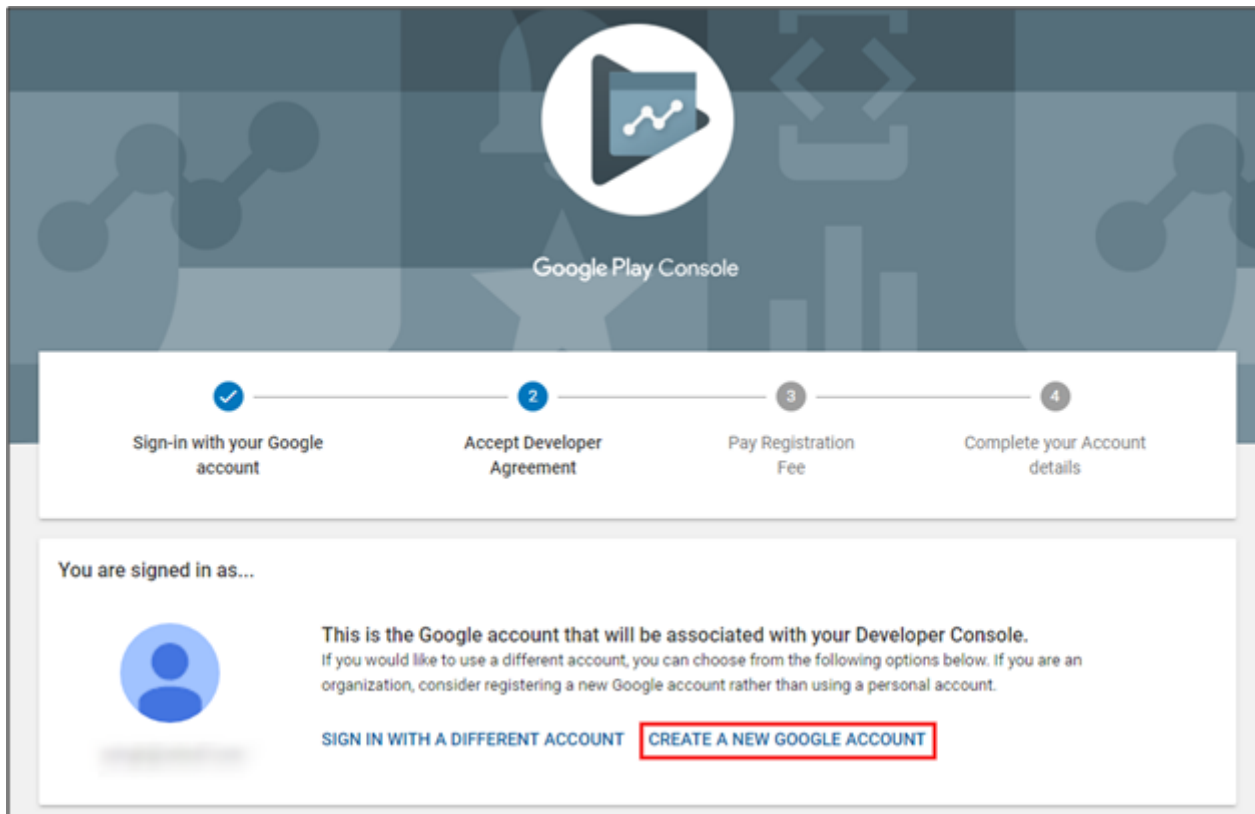
Output :



**Practical 28 : Demonstrate publishing an Android app on the Google Play Store following the policies and guidelines.**

### **Step 1 – Google Play Console Developer Account.**

One of the primary steps of publishing an Android application on the Play Store is creating a Google developer account on the Google Play Console. Google Play Console can be understood as a backend operating system for all the apps published on the Play Store. Developers who intend to publish an app on the Play Store must create a developer account on the console and pay a one-time fee of \$25, payable using a credit card or through online banking methods. Also, remember that post submission, the developer account can take up to 48-hours to activate.



You'd have to sign in using your Gmail IDs as soon as you enter the console. Post your sign-in; the Google Play Console will enquire whether an individual developer or organization will own the app account. And, based on the ownership, you will enter basic information about yourself or your organization which will lead to the payment section and the final account creation.

Finally, ensure you fill out all the credentials asked while creating the account.

### **Step 2 – Set-up up a Google Merchant Account.**

If your app involves in-app purchases, the next essential step is to link your developer account with your merchant account. If you already have an existing merchant account, you can navigate to Download reports and select Financial. However, to access this page, you must create a Google Merchant Account. And to create one, click on setup up a merchant account.



## Your organization

This information won't be shown on Google Play

|                           |                                                                                                                                                                                     |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Organization name         | Enter the full, legal name of your organization. For example Google LLC.<br><input type="text"/>                                                                                    |
| Organization type         | Any information we ask for to verify your organization will be tailored to your organization type<br><input type="text" value="Select your organization type"/>                     |
| Organization size         | Select the number of employees in your organization<br><input type="text" value="Select number of employees"/>                                                                      |
| Organization address      | <input type="text" value="Country"/><br><small>Select country or region</small>                                                                                                     |
| Organization phone number | Enter the main phone number associated with your organization<br><input type="text"/><br><small>Include the + symbol, country code, and area code.</small>                          |
| Organization website      | Enter the URL of your organization's main website<br><input type="text" value="Enter a URL starting with http:// or https://"/><br><input type="checkbox"/> We don't have a website |

[Back](#)[Next](#)

- Policy status
- Users and permissions
- Order management
- Download reports**
- Reviews
- Statistics
- Financial**
- Account details
- Developer page
- Associated developer accounts
- Activity log
- Setup
- Email lists

## Download financial reports

### Download data for offline evaluation

Download CSV files of key data from across your apps including reviews, installs and updates, and financial reports. Get and use your Google Cloud Storage directory paths to access to data programmatically.



You need to set up a Google Payments merchant account to access this page

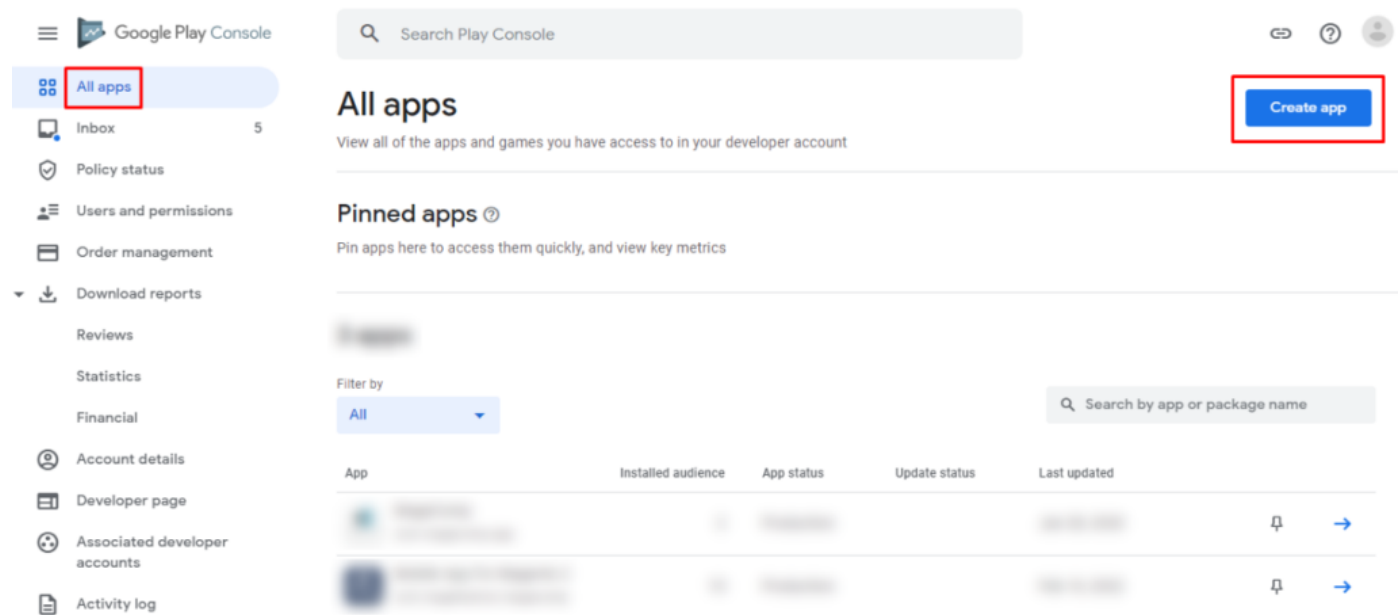
[Set up a merchant account](#)

Once you create your new merchant account, it will automatically get linked to your Google Play Console account and enable you to monitor, manage, and analyze the app sales and generate reports.

### Step 3 – Create Application

Once the merchant account is linked to your Google Play Console, the next step is to create an application. And for creating an application, there are a few essential steps that you need to follow:

- Click on – Menu > All applications
- Select the ‘Create Application’ option.



Next, the play console will ask you to enter some basic app details. For instance,

- App Name – You must enter a 30-character long name in this field which will be displayed on the Google Play Console. However, this app name can be changed afterward.
- Default language – Another essential field is the setup of the app language. You can navigate to the drop-down menu and set a default language for your app.
- App or game – The next step is to define whether you upload an app or a game, but this can again be revised afterward from the store settings.
- Free or paid – Define whether your app will be available free of cost or will require the user to pay for it. The free or paid section can be updated from the Paid app page later, but only until you publish your app. Once the app is live, you cannot transform your app from free to paid.

Once all of the above information is filled and verified, the Google Play Console will enquire for affirmations from you. Ensure that your app matches the Google policies of the Developer Program and Accepts US export laws. As soon as you agree to the terms and conditions, click

# Create app

## App details

App name

This is how your app will appear on Google Play

0 / 30

Default language

English (United States) – en-US

App or game

You can change this later in Store settings

☐ App

☐ Game

Free or paid

You can edit this later on the Paid app page

☐ Free

☐ Paid

## Create App.

### Step 4. App Store Listing Details

The next step to uploading the app on the Google Play console is filling in the essential information regarding the application listing. Once you click on the 'Create App,' the Play Console will automatically take you to a consolidated dashboard wherein you'll have to enter the necessary details to set up your app.

Dashboard

## Set up your app



### Provide information about your app and set up your store listing

Let us know about the content of your app, and manage how it is organized and presented on Google Play

Hide tasks ^

#### LET US KNOW ABOUT THE CONTENT OF YOUR APP

- ☐ App access >
- ☐ Ads >
- ☐ Content rating >
- ☐ Target audience >
- ☐ News apps >
- ☐ COVID-19 contact tracing and status apps >
- ☐ Data safety >

#### MANAGE HOW YOUR APP IS ORGANIZED AND PRESENTED

- ☐ Select an app category and provide contact details >
- ☐ Set up your store listing >

The Google Play Console will enquire about the following:

- App name – As you have already entered the app name in the previous step, you need not enter the same name again; however, if you wish to revise the title, this is where you can change the name.

- Short description – This field requires you to enter an 80-character-long description that best describes your app.
- Full description – The following field to the short description enables you to explain your app in detail. You can expand the word limit to 4000 characters and leverage your targeted keywords to lead Google to share your app with the relevant audience.

## Main store listing

Default – English (United States) – en-US [Manage translations](#) ▼

### App details

Check the [Metadata policy](#) and [Help Center guidance](#) to avoid common issues with your store listing. Review all [program policies](#) before submitting your app.

If you're eligible to [provide advance notice](#) to the app review team, contact us before publishing your store listing.

App name \*

This is how your app will appear on Google Play 7 / 30

Short description \*

A short description for your app. Users can expand to view your full description. 0 / 80

Full description \*

0 / 4000

Once all this information is added to the Google console, the next step is adding the app graphics, category of the app, and the privacy policy. Remember, we asked you to keep high-quality images ready before beginning the app publishing process; this is precisely where all those images will be leveraged.

Further, here are the details you would require:

| Particulars                         | Details                                                                                                                                                                              |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Screenshots                         | -2 to 8 in number, JPG, or PNG. The ratio shouldn't exceed 2:1                                                                                                                       |
| Icon                                | – 512 X 512– PNG– Maximum file size: 1024KB                                                                                                                                          |
| Localization                        | -If your app comes in several languages, you need to mention them and add additional translations of your app's information to appeal to the users coming and checking out your app. |
| Application type and categorization | – Navigate to the Drop-down menu and select application type – game or app.- Pick a category suitable for your app- Rate your content after uploading your APK.                      |
| Contact details                     | -Provide the necessary contact forms so users can contact you.                                                                                                                       |
| Privacy Policy                      | -To escape the breach of app privacy, Google mandates adding a privacy policy while publishing your app. -If you need a break, click Save Draft and complete it later.               |

Once you are done uploading details, Hit the Save button.

## Step 5 – Content Rating

The next most crucial step is the content rating questionnaire. Without rating your app's content, Google will consider it an Underrated app and will certainly remove it from Google Play Store. And since you might not want it, let's learn the steps of adding a content rating.

To add the content rating, you'll have to navigate to the main dashboard, set up your app, and select the Content rating option.

Dashboard

### Set up your app



#### Provide information about your app and set up your store listing

Let us know about the content of your app, and manage how it is organized and presented on Google Play

Hide tasks ^

#### LET US KNOW ABOUT THE CONTENT OF YOUR APP

- App access >
- Ads >
- Content rating >
- Target audience >
- News apps >
- COVID-19 contact tracing and status apps >
- Data safety >

The Next dashboard will pop up, and you'll be able to navigate the "Start Questionnaire" button; you have to click the tab and get started.

## Content ratings

### Receive ratings for your app from official rating authorities

Complete the content rating questionnaire to receive official content ratings for your app. Ratings are displayed on Google Play to help users identify whether your app is suitable for them.



Start questionnaire

[Learn more](#)

You'll have to enter basic information about your app in the content rating section. This section is divided into three sub-sections – Categories, Questionnaire, and Summary.

In the Category section, you have to enter the email address that the users can leverage to contact you and the category of the app you are publishing on the Play Store.

## Content ratings

[Discard changes](#)

1 Category — 2 Questionnaire — 3 Summary

### Category

Email address

This will be used to contact you about your content ratings. It may be shared with rating authorities and IARC.

Category

- ☐ Game  
The app is a game or betting app. Examples include: Candy Crush Saga, Temple Run, Mario Kart, The Sims, Angry Birds, casino games, or daily fantasy sports.
- ☐ Social or Communication  
The primary purpose of this app is to meet or communicate with people. Examples include Facebook, Twitter, Skype, and SMS.
- ☐ All Other App Types  
Any app that isn't a game, social networking app, or communication app. Examples include entertainment products, consumer stores, news apps, lifestyle apps, streaming services, utilities, tools, emoji sets, fitness apps, magazines, and customizations.

Once you are done filling the above fields, click the Next button, and you'll be redirected to the questionnaire section. The questionnaire section lets Google explore more about your app to understand your target audience better.

## Content ratings

[Discard changes](#)

✓ Category — 2 Questionnaire — 3 Summary

### All Other App Types

Downloaded App  Completed

Does the app contain any ratings-relevant content (e.g., sex, violence, language) downloaded as part of the app package (code, assets)? [Learn more](#)

☐ Yes ☒ No

User Content  Completed

Does the app natively allow users to interact or exchange content with other users through voice communication, text, or sharing images or audio? [Learn more](#)

☐ Yes ☒ No

Once all the details are filled in, you can look at the content rating summary and hit 'Submit' to apply the changes.

### Step 6 – Create & Upload Android App to Google Play

Uploading the APK to the Google Play Console is the foremost step of the app publishing process, where the app is finally uploaded and submitted for Google to review and go live.

But before we go and upload the app, one crucial decision you need to make is the type of app release – Internal Test (available for up to 100 testers that you pick), Close Test (available to just a specific number of testers), Production Release (available to all the Play store users in your chosen countries), or Open Test (available to Google Playtesters, and users to join tests from your store listing).

Once you have decided on the testing, you can go to the dashboard and select **“Create a new release.”**



### Publish your app on Google Play

Publish your app to real users on Google Play by releasing it to your production track

Hide tasks ^

- Select countries and regions >
- CREATE AND ROLL OUT A RELEASE**
  - Create a new release >**
  - Review and roll out the release

Post selecting “**Create a new release**,” you’ll be redirected to a dashboard, wherein you will upload the app bundles and the release details.

#### Create production release

##### App bundles



Drop app bundles here to upload

[Upload](#) [Add from library](#)

##### Release details

Release name \*

0 / 50

This is so you can identify this release, and isn't shown to users on Google Play. We've suggested a name based on the first app bundle or APK in this release, but you can edit it.

Once you enter all the details, confirm that everything is correct, and take the last step of this guide on Google Play app upload and add the application to the platform. Then, navigate to the ‘App Releases’ tab and select ‘Manage Production’ followed by ‘Edit Release’; click on ‘Review’ and then the ‘Start rollout to production’ option.

Select the ‘Confirm’ option, and that’s it!

You are done with the successful upload of your app to the Google Play Store account for free.

Once the Google Play app upload is done, you must patiently wait for your application to get reviewed and approved by Google. The Google app approval process can take a few hours or extend up to 7 days, so ensure that you are well prepared for both successful publishing of the app or for doing any revisions if required.